

Model Tuning & Cross-Validation

A practical guide to hyperparameter tuning, K-Fold cross-validation, Grid Search, Random Search, and avoiding overfitting.

Why this matters

A machine learning model can perform well on training data but poorly on unseen data. Model tuning helps choose better hyperparameters, while cross-validation provides a more reliable estimate of how the model is likely to generalize. Together, they form a core part of a robust ML workflow.

1. What is Model Tuning?

Model tuning is the process of selecting hyperparameters that give a model the best validation performance. Hyperparameters are settings chosen before training; they are different from model parameters learned from the data.

Model	Example hyperparameters
KNN	n_neighbors, weights, metric
Decision Tree	max_depth, min_samples_split, criterion
Random Forest	n_estimators, max_depth, max_features
SVM	C, kernel, gamma
Logistic Regression	C, penalty, solver

Parameters vs Hyperparameters

Parameters are learned during training (for example, regression coefficients). **Hyperparameters** are selected by the practitioner or tuning procedure (for example, K in KNN or tree depth).

2. What is Cross-Validation?

Cross-validation repeatedly splits the available training data into training and validation portions. It helps estimate generalization performance without relying on a single train/validation split.

K-Fold Cross-Validation

In K-Fold CV, the training data is divided into **K** approximately equal folds. The model is trained K times. Each run uses one fold for validation and the remaining K-1 folds for training. The final score is commonly the mean of the K validation scores.

Example: 5-Fold CV

- Fold 1 → validate on Fold 1, train on Folds 2-5
- Fold 2 → validate on Fold 2, train on Folds 1, 3-5
- Fold 3 → validate on Fold 3, train on Folds 1-2, 4-5
- Fold 4 → validate on Fold 4, train on Folds 1-3, 5

Fold 5 → validate on Fold 5, train on Folds 1–4
Final CV score = mean of the five validation scores.

Why K-Fold is useful

- Uses data more efficiently than a single validation split.
- Shows how performance changes across different subsets.
- Reduces dependence on one lucky or unlucky train/validation split.
- Can be used inside hyperparameter search.

3. Model Tuning + Cross-Validation

The important idea is to tune hyperparameters using only the training data. Cross-validation can evaluate each candidate hyperparameter combination, and the best combination is selected according to the chosen scoring metric.

Typical workflow

Step	Action
1	Split data into training and final test sets.
2	Define a model and candidate hyperparameters.
3	Run K-Fold cross-validation on the training set.
4	Compare mean CV scores.
5	Select the best hyperparameters.
6	Refit the best model on the full training set.
7	Evaluate once on the untouched test set.

4. Grid Search

GridSearchCV tries every combination in the parameter grid and evaluates each combination using cross-validation. It is systematic and easy to understand, but can become expensive when the grid is large.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 5, 10],
    "min_samples_split": [2, 5]
}

grid = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)
```

5. Randomized Search

RandomizedSearchCV samples a fixed number of hyperparameter combinations instead of trying every combination. This can be much faster for large search spaces and is often a strong baseline for tuning.

```
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    model,
    param_distributions=param_grid,
    n_iter=20,
```

```
cv=5,  
scoring="accuracy",  
random_state=42,  
n_jobs=-1  
)  
  
search.fit(X_train, y_train)  
  
print(search.best_params_)
```

Grid Search vs Random Search

Feature	Grid Search	Random Search
Search strategy	All combinations	Randomly sampled combinations
Cost	Can become expensive	Usually more controllable
Best for	Small, focused search spaces	Large search spaces
Main control	Parameter grid	n_iter + distributions

6. K-Fold Variations

Stratified K-Fold keeps class proportions approximately consistent across folds. It is commonly preferred for classification, especially when classes are imbalanced.

Time Series Split is designed for ordered data where future observations must not be used to predict the past. Ordinary random K-Fold can cause temporal leakage in time-series problems.

Group K-Fold keeps samples from the same group together, which is useful when multiple rows belong to the same person, patient, customer, or other entity.

7. Choosing K

Common choices include **K = 5** or **K = 10**. A larger K gives each model more training data but requires more model fits. The right value depends on dataset size, compute budget, variance, and the problem.

8. Avoid Data Leakage

A major rule: preprocessing steps that learn from data should be fitted inside each training fold, not on the entire dataset before cross-validation. For example, scaling or imputation before CV can leak information from validation folds.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

params = {
    "model__C": [0.01, 0.1, 1, 10]
}

search = GridSearchCV(pipe, params, cv=5, scoring="accuracy")
search.fit(X_train, y_train)
```

9. Common Mistakes

- **Tuning on the test set:** this makes the test set part of model selection and gives an optimistic estimate.
- **Scaling before CV:** fit the scaler within a Pipeline so each fold learns preprocessing from its training portion.
- **Using the wrong metric:** accuracy may be misleading for imbalanced classification; consider precision, recall, F1, ROC-AUC, or PR-AUC.
- **Huge parameter grids:** more combinations are not automatically better; start focused and expand intelligently.
- **Ignoring variance:** compare the mean CV score and its spread, not just one fold's score.

10. A Practical ML Tuning Template

```
# 1. Hold out the test set
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Build pipeline
pipe = Pipeline([
    ("preprocess", StandardScaler()),
```

```

("model", LogisticRegression(max_iter=1000))
])

# 3. Tune with CV
search = GridSearchCV(
    pipe,
    {"model__C": [0.01, 0.1, 1, 10]},
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

search.fit(X_train, y_train)

# 4. Final evaluation on untouched test data
y_pred = search.predict(X_test)
print(search.best_params_)
print(accuracy_score(y_test, y_pred))

```

Key Takeaways

- ✓ Hyperparameters control model behavior and should be selected systematically.
- ✓ K-Fold cross-validation gives a more stable estimate than a single validation split.
- ✓ GridSearchCV exhaustively checks a defined grid; RandomizedSearchCV samples candidates.
- ✓ Keep the final test set untouched until model selection is complete.
- ✓ Use Pipelines to prevent preprocessing leakage during cross-validation.
- ✓ Choose the scoring metric based on the business and ML objective—not habit.

Suggested GitHub topics: model tuning · hyperparameter optimization · K-Fold CV · GridSearchCV · RandomizedSearchCV · StratifiedKFold · data leakage · sklearn Pipeline · model evaluation